

MRES LATE STAGE GATEWAY REPORT

IMPERIAL COLLEGE LONDON

DYSON SCHOOL OF DESIGN ENGINEERING

Human-Robot Collaboration through VLA-based Learning

Author:

Saniya Patwardhan (CID: 06059013)

Research Supervisor:

Assoc. Prof. Petar Kormushev

Date: July 22, 2026

I confirm that this report is my own original work and has not been copied or submitted elsewhere. Any use of AI tools has been appropriately acknowledged and has not replaced my own intellectual contribution. All sources have been properly referenced

Human-Robot Collaboration using VLA-based Learning

Saniya Patwardhan
MRes Late Stage Report

Abstract—Human-robot collaboration requires robots to understand high-level goals while performing reliable manipulation. Although Vision-Language-Action (VLA) models enable end-to-end mapping from visual observations and language instructions to robot actions, adapting them to novel robot embodiments remains challenging.

This project investigates the adaptation of the $\pi 0.5$ Vision-Language-Action model to a Baxter robot in a custom MuJoCo environment. A demonstration collection pipeline, LoRA-based fine-tuning framework, and real-time inference system were developed to evaluate multiple policy variants. Experimental findings show that transitioning from velocity-control to position-control improved manipulation performance and highlighted the importance of train-inference consistency during VLA adaptation.

A Vision-Language Model (VLM) planner generates manipulation sequences from current and goal workspace configurations. As planner development was presented in the Early Stage Gateway report, this report focuses on the development and evaluation of the fine-tuned VLA policy.

Project updates, demonstrations, and source code are available at <https://saniya2912.github.io/pi05-baxter/>.

I. LITERATURE REVIEW

Human-robot collaboration (HRC) aims to enable humans and robots to work together by leveraging their complementary capabilities [1]. The emergence of collaborative robots has enabled safe operation in shared workspaces without physical barriers [2]. While these systems have improved physical interaction, many still rely on separate perception, planning, and control mod-

ules, limiting their ability to adapt to changing task goals.

Recent work has addressed different aspects of collaborative reasoning. SAGE [3] estimates human actions and gaze, Therblig-based methods [4] represent manipulation as structured primitives, and long-horizon planning approaches [5] decompose complex tasks into executable action sequences. Visual instruction understanding has also been demonstrated using LEGO assembly benchmarks [6], [7]. Although effective, these methods typically depend on handcrafted task representations or specialised pipelines, limiting generalisation.

Prior to emergence of Vision-Language-Action (VLA) models, systems such as SayCan [8] combined large language models with predefined robotic skills for language-guided planning. While effective for structured tasks, they relied on manually designed skill libraries rather than end-to-end visuomotor learning.

Recent VLA models instead learn a direct mapping from visual observations and language instructions to robot actions. RT-1 [9] demonstrated large-scale robotic learning, while RT-2 [10] incorporated vision-language pretraining to improve semantic reasoning. Open-source models such as OpenVLA [11] and SmolVLA [12] have made VLA research more accessible, while the state-of-the-art $\pi 0$ [13] and $\pi 0.5$ [14] models introduce flow matching to predict chunks of future actions, improving continuous manipulation. However, most VLA research focuses on policy learning rather than adaptation to new robot embodiments or integration with high-level planning.

Natural language has also become an important interface for human–robot interaction. Wang *et al.* [15] proposed a framework for modifying robot trajectories through language, while VLA-bot [16] combines multimodal perception with manipulation for collaborative assembly tasks. However, neither investigates efficient adaptation of large pretrained VLA models to novel robotic platforms.

Overall, recent advances in multimodal robotics have significantly improved robot perception, planning, and manipulation. Vision–Language–Action models show particular promise as general-purpose robotic policies, motivating further investigation into their adaptation and integration within collaborative robotic systems.

II. RESEARCH GAP

Despite recent advances in Vision–Language–Action (VLA) models, three key challenges remain. First, efficient adaptation study of pretrained VLA models to novel robot embodiments with different kinematics, camera viewpoints, and control interfaces remains underexplored. Second, high-level task planning and low-level manipulation are typically addressed separately, with few hierarchical frameworks integrating Vision–Language Models (VLMs) and VLA policies. Finally, limited attention has been given to the engineering decisions that influence VLA adaptation, including action representation, demonstration generation, and train–inference consistency.

This research addresses these challenges by adapting the $\pi 0.5$ Vision–Language–Action model to a Baxter robot, evaluating key engineering decisions during fine-tuning, and integrating the resulting manipulation policy with a Vision–Language planner for goal-directed task execution.

A. Research Questions

- 1) How effectively can a pretrained Vision–Language–Action model be adapted to a novel robot embodiment?
- 2) How do engineering decisions, including action representation, demonstration genera-

tion, and train–inference consistency, affect manipulation performance?

- 3) Can a Vision–Language Model be integrated with a fine-tuned Vision–Language–Action policy to achieve goal-directed manipulation?

III. METHODOLOGY

A. Overall System Architecture

The proposed framework (Fig. 1) follows a hierarchical architecture consisting of a Vision–Language Model (VLM) planner and a Vision–Language–Action (VLA) policy.

The Gemma-3-12B VLM compares the current and goal workspace images to generate an ordered sequence of manipulation primitives. Each primitive is provided as a language instruction to a fine-tuned $\pi 0.5$ policy, which generates robot actions from visual observations and robot state information. The actions are executed within a MuJoCo simulation environment. Separating planning from execution enables independent evaluation of each component and supports future replanning when the workspace changes.

B. Choice of Platform

1) *Robot DE NIRO*: Robot DE NIRO (Fig. 2) was selected as the long-term deployment platform due to its Baxter dual-arm configuration, mobile base, and human-centred design, making it well suited to future human–robot collaboration research. Although this work focuses on simulation, the system was developed with future deployment on DE NIRO in mind.

2) *MuJoCo Simulation*: Development was performed in the MuJoCo physics simulator to enable rapid experimentation, safe debugging, and automated demonstration collection. Simulation also allowed train–inference mismatches to be identified and resolved before deployment, providing a reliable environment for policy development and evaluation.

3) $\pi 0.5$: The $\pi 0.5$ [14] Vision–Language–Action (VLA) model released by Physical Intelligence

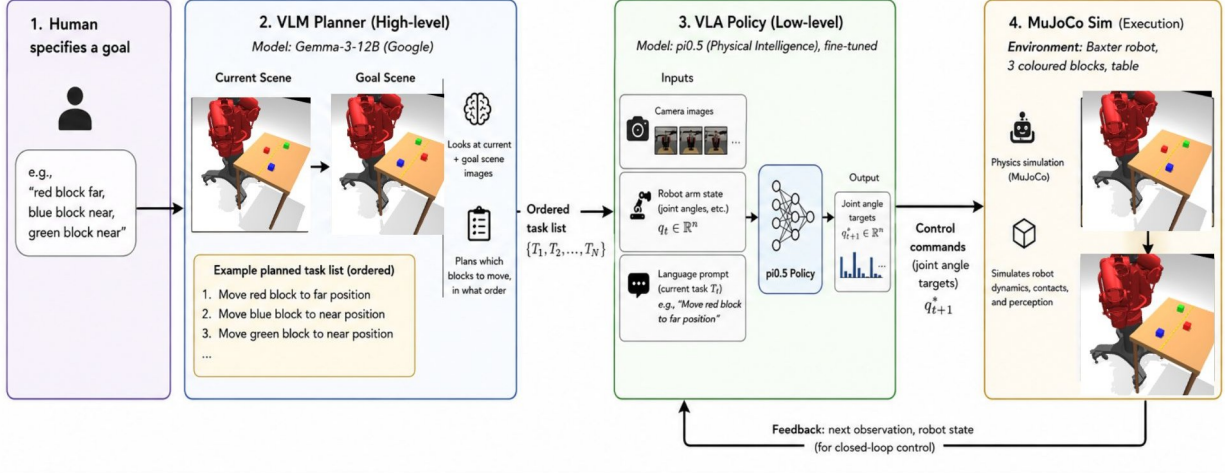


Fig. 1: Proposed system architecture.



Fig. 2: Design Engineering’s Natural Interaction Robot (DE NIRO).

was selected as the foundation policy for this research. The model combines visual observations, robot state information, and natural language instructions to generate robot actions directly from high-level task descriptions.

Several factors motivated its selection. First, $\pi 0.5$ has demonstrated strong manipulation performance across multiple robotic embodiments. Second, the release includes a training and inference framework that can be adapted to custom robotic

platforms. Finally, the model supports parameter-efficient fine-tuning, making experimentation feasible within the computational resources available for this project.

Compared to other open VLA models such as OpenVLA, $\pi 0.5$ employs a flow-matching architecture that predicts action chunks rather than individual actions. This enables more expressive trajectory modelling and was considered particularly suitable for the sequential manipulation tasks investigated in this work. A comparison between the two approaches is shown in Table I.

TABLE I: Comparison of OpenVLA and $\pi 0.5$

OpenVLA	$\pi 0.5$
Open-source model	Research release
Autoregressive actions	Flow-matching actions
Single-step prediction	Action chunk prediction
Simpler training paradigm	Advanced trajectory modelling
Easier deployment	Stronger manipulation modelling

C. Vision-Language-Action Adaptation Strategy

1) *Need for Fine-Tuning:* Although Vision-Language-Action (VLA) models are pretrained on diverse robots and manipulation tasks, direct deployment on a new robot is challenging

due to differences in kinematics, camera placement, workspace configuration, and control interfaces. The Baxter platform and structured block-manipulation tasks introduced in Section III-D1 differ substantially from the original $\pi 0.5$ training data, making fine-tuning necessary. The objective is therefore to adapt the pretrained policy to the target robot and task using a relatively small demonstration dataset rather than training a policy from scratch.

2) *Parameter-Efficient Adaptation using LoRA*: Low-Rank Adaptation (LoRA) was used to fine-tune the pretrained $\pi 0.5$ model. Instead of updating all 3 billion parameters, LoRA freezes the pretrained model and trains low-rank adapters within the attention layers.

Adapters were applied to both the 2B-parameter PaliGemma vision-language backbone and the 300M-parameter action expert. Using a rank of 16 resulted in approximately 30–50 million trainable parameters and reduced GPU memory usage to around 17 GB, enabling training on a single RTX 5090 GPU.

LoRA enabled efficient evaluation of multiple datasets, action representations, and state configurations while preserving the pretrained model’s visual and language capabilities.

D. Development of the Demonstration Collection Pipeline

A demonstration collection pipeline was developed to generate task-specific data for fine-tuning the $\pi 0.5$ model. Demonstrations were generated through scripted execution in MuJoCo, providing repeatable and scalable data collection. The pipeline consisted of four stages: task design, inverse kinematics trajectory generation, demonstration recording, and conversion to the LeRobot dataset format.

1) *Task Design*: The target task is to transform an initial block arrangement into a user-specified goal configuration. To simplify learning, the problem was decomposed into six reusable manipulation primitives involving three coloured

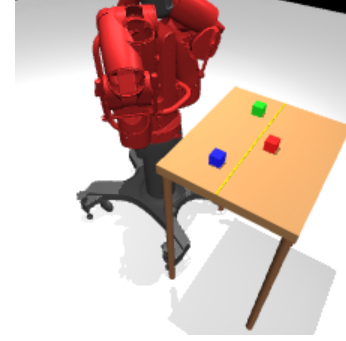


Fig. 3: Tabletop workspace for execution of primitive tasks

blocks placed on a tabletop divided into near and far regions (Fig. 3).

- Move the red block to the near side
- Move the red block to the far side
- Move the blue block to the near side
- Move the blue block to the far side
- Move the green block to the near side
- Move the green block to the far side

These primitives can be combined by the VLM planner to perform more complex rearrangement tasks.

2) *Damped Least Squares Inverse Kinematics*: To generate demonstrations, a controller capable of reliably executing pick-and-place trajectories was required.

For a desired end-effector displacement Δx , the corresponding joint update Δq is computed as

$$\Delta q = J^T(JJ^T + \lambda^2 I)^{-1} \Delta x \quad (1)$$

where J is the manipulator Jacobian and λ is the damping coefficient.

DLS inverse kinematics was selected for its numerical stability near singular configurations and its compatibility with both simulation and future deployment on the physical robot.

3) *Scripted Demonstration Collection*: Demonstrations were generated autonomously through a scripted pick-and-place controller implemented within the MuJoCo simulation environment.

Each episode followed a structured sequence consisting of:

- 1) Move to a pre-grasp position above the target block.
- 2) Descend towards the block.
- 3) Close the gripper and establish grasp contact.
- 4) Lift the object above the workspace.
- 5) Transport the object to the target region.
- 6) Lower the object and release the grasp.
- 7) Return to a neutral configuration.

Scripted execution ensured consistent demonstrations and enabled rapid dataset generation without teleoperation. To improve generalisation, block positions were randomised at the start of each episode while maintaining task feasibility. A total of 600 demonstrations were collected, with 100 demonstrations for each primitive task.

4) *Dataset Processing*: Recorded demonstrations were converted to the LeRobot dataset format required by the $\pi 0.5$ training framework. Each sample contained workspace and wrist camera images, robot state information, a language instruction, and the corresponding action target. Dataset validation verified trajectory integrity and observation–action consistency before fine-tuning.

E. Design of the Action Representation

Two action representations were evaluated. Initial experiments used the velocity-control formulation of the original $\pi 0.5$ framework, in which the policy predicts joint velocities. This was later replaced with position control, where the policy predicts target joint positions and a proportional controller generates the corresponding robot motions. The transition required new demonstrations with future joint configurations as action targets. Its impact on manipulation performance is evaluated in Section V.

F. Fine-Tuning and Inference Pipeline

The collected demonstrations were used to fine-tune the pretrained $\pi 0.5$ model using the OpenPI

training framework and datasets formatted according to the LeRobot specification.

1) *Training Configuration*: Each training sample consisted of workspace and wrist camera images, robot state information, and a language instruction. The model was trained to predict the corresponding action sequence using LoRA adapters, while the pretrained backbone remained frozen. Multiple training runs were performed to evaluate different action representations, datasets, and state configurations, with checkpoints saved every 10,000 training steps.

2) *Inference Architecture*: A two-process inference framework was developed consisting of a GPU-based policy server and a MuJoCo simulation client communicating via WebSocket. At each timestep, the client sends camera observations, robot state information, and the task instruction to the policy server. The server applies the same preprocessing used during training, performs inference with the fine-tuned $\pi 0.5$ model, and returns an action chunk for execution.

For position-control policies, predicted joint positions are executed by a proportional controller within MuJoCo. This framework provides a consistent environment for checkpoint evaluation and policy comparison.

TABLE II: Fine-tuning configuration

Model	$\pi 0.5$
Adaptation	LoRA
Framework	OpenPI
Dataset	LeRobot
Training Steps	Various (100k-500k)
Checkpoint Interval	10k
Hardware	RTX 5090 (32 GB)

G. Vision-Language Planner

A Vision-Language Model (VLM) was introduced to provide high-level task planning within the proposed human-robot collaboration framework. While the fine-tuned $\pi 0.5$ policy executes individual manipulation primitives, it has no knowledge of the desired final scene configuration. The VLM planner addresses this limitation by converting a

goal scene into an ordered sequence of manipulation primitives (Fig. 4).

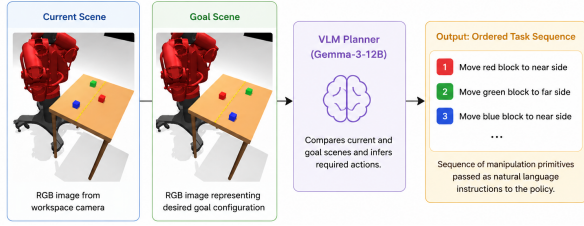


Fig. 4: High-level VLM planning pipeline.

1) *Planner Implementation*: Gemma-3-12B was selected due to its multimodal reasoning capabilities and ability to jointly process images and text. The planner receives the current workspace image together with a goal-state image and identifies the differences between the two scenes. Prompt engineering constrains the model to output only the manipulation primitives listed in Table III.

To improve reliability, each coloured block is evaluated independently rather than reasoning about all objects simultaneously. For every mismatch between the current and goal scene, the planner generates the corresponding manipulation primitive. These primitives are then passed sequentially to the fine-tuned $\pi 0.5$ policy as natural language instructions.

TABLE III: Manipulation primitives used by the planner

Object	Primitive Actions
Red	Move Near, Move Far
Blue	Move Near, Move Far
Green	Move Near, Move Far

This hierarchical decomposition enables the robot to perform object arrangements that were never explicitly demonstrated during training.

H. Evaluation Methodology

The final objective of this research is to develop a hierarchical human-robot collaboration system capable of transforming an initial workspace into a user-specified goal configuration. Since this capability depends on multiple intermediate components, evaluation was performed at three levels: primitive skill success, manipulation success, and collaborative task success.

1) *Primitive Skill Success*: Primitive skill success evaluates whether the policy performs the requested task. A trial is considered successful if the specified coloured block reaches the correct side of the workspace following the language instruction. This metric measures task understanding without considering the quality of manipulation.

2) *Manipulation Success*: Manipulation success evaluates execution quality. A successful trial requires the robot to complete the full pick-and-place sequence:

- 1) Approach the target object.
- 2) Grasp the object.
- 3) Lift the object.
- 4) Transport it to the target region.
- 5) Release the object.

Unlike primitive skill success, this metric excludes accidental task completion caused by pushing or unintended collisions.

3) *Collaborative Task Success*: Collaborative task success represents the final evaluation objective of the project. Starting from an initial workspace configuration, the complete planner-policy system must rearrange all objects to match a user-specified goal configuration. Success therefore depends on correct task planning and successful execution of every manipulation primitive.

Evaluation of dynamic human intervention and replanning will be completed during the final stage of the project.

4) *Checkpoint Comparison*: Throughout development, multiple policy checkpoints (refer Table IV) were trained using different action representations, datasets, and state observations. Comparative evaluation was performed through rollout analysis and task success assessment to determine the impact of each engineering decision. Particular emphasis was placed on comparing velocity-control and position-control policies, with the resulting findings presented in Section V.

TABLE IV: Policy checkpoints evaluated throughout development.

Checkpoint	Steps	Key change
pi05_base	—	Zero-shot; no fine-tuning
vel_100k	100k	Velocity control; 4 tasks
pos_200k	200k	Position control; 6 tasks
pos_v2	500k	Native 10 Hz; 11-dim state
pos_v3	+200k	Warm-start v2; $\times 2.5$ near demos

Note: 4 tasks = Tasks with only red and blue blocks.

IV. EXECUTION AND DESIGN ENGINEERING APPLICATION

The project followed an iterative engineering design process in which experimental findings continuously informed subsequent system development. Rather than implementing the complete collaborative framework in a single stage, individual components were developed, evaluated, and refined before integration.

Key engineering contributions included the development of a custom MuJoCo simulation environment, a scripted Damped Least Squares inverse kinematics data collection pipeline, adaptation of the $\pi 0.5$ Vision-Language-Action model through LoRA fine-tuning, redesign of the action representation from velocity to position control, and implementation of a hierarchical Vision-Language planner for task sequencing. Each component was validated independently before integration into the complete planner-policy framework.

This incremental development strategy reduced hardware risk, enabled rapid experimentation, and allowed individual design decisions to be evaluated systematically before progressing towards end-to-end human-robot collaboration.

V. PRELIMINARY FINDINGS AND INSIGHTS

A. Velocity-Control Policy Evaluation

Initial fine-tuning experiments employed a velocity-control action representation in which the policy predicted joint velocity commands. The policy demonstrated clear language grounding by consistently approaching

the correct coloured block, indicating that it had learned the relationship between visual observations, language instructions, and task objectives.

However, reliable manipulation was not achieved. The **vel_100k** checkpoint achieved a 0% success rate across 40 evaluation trials and failed to lift the block in any trial (Table V). Mean gripper close steps of 762 and 940 for the red-far and red-near tasks indicate that the gripper remained open throughout most of the episode, causing the robot to push the block rather than grasp it.

These results suggested that the policy had learned *what* task to perform but not *how* to execute it. This motivated further investigation into train–inference mismatches and the subsequent transition to position-control actions.

Analysis indicated that the poor performance could not be explained solely by the action representation, motivating investigation of the complete training and deployment pipeline.

B. Analysis of Train–Inference Mismatches

A significant part of the project involved identifying train–inference mismatches that limited policy performance. Four major issues were identified and resolved.

The first involved incorrect action targets during dataset generation. Actions were initially recorded as small position deltas derived from velocity commands, producing target magnitudes of approximately 0.001 rad after downsampling. Consequently, the policy generated negligible arm motion during inference (mean action magnitude ~ 0.001 rad compared to the expected 0.05–0.3 rad). This was resolved by recording the desired joint configuration at the subsequent timestep rather than position deltas.

A second issue was a grasp alignment error. During demonstration collection, the gripper consistently stopped 2–3 cm above the block due to an overly conservative descent tolerance. Diagnostic logging showed a vertical error of 28–30 mm. Reducing the descent target to $z_{\text{block}} - 0.010$ m

with a tolerance of 0.010 m reduced the error to within 1 mm.

Two deployment-specific issues were also identified:

- **Gripper initialisation mismatch:** After simulator reset, the gripper control signal remained partially closed (`gripper_norm` $\approx$ 0.65), causing the policy to immediately enter carry mode. Resetting the control signal to the open position restored the expected initial state.
- **Gripper hysteresis:** The flow-matching policy occasionally generated close–open–close commands within a single action chunk, causing dropped objects during transport. This was resolved by latching the gripper closed after a successful grasp and releasing only when a placement command was detected.

These observations demonstrate that train–inference consistency is as important as the training configuration itself. Figure ?? illustrates the gripper state before and after the initialisation fix.

C. Position-Control Policy Evaluation

To address the limitations observed in velocity-control experiments, the action representation was redesigned to predict target joint positions rather than joint velocities. Under this formulation, a proportional controller converts predicted targets into executable velocity commands:

$$v = \text{clip}(K_P(q_{\text{target}} - q), -v_{\text{max}}, v_{\text{max}}) \quad (2)$$

with $K_P = 40$ and $v_{\text{max}} = 1.5$ rad/s. This separates the *where* (learned by the policy) from the *how fast* (handled by the controller), substantially simplifying the learning problem by allowing the policy to predict desired configurations while low-level control is handled separately. In practice, smoother trajectories, more stable object approach motions, and more consistent grasp attempts. The `pos_200k` checkpoint achieved 20% trial-level success (12/60 trials) compared to 0% for the velocity-control baseline, with the block-lifted rate rising from 0% to 10–30% across tasks

that had previously shown no successful manipulation. Position-control policies were therefore adopted as the standard representation for all subsequent training runs.

D. Checkpoint Comparison and Policy Evolution

Five policy checkpoints were evaluated across the full six-task protocol (10 trials per task, 60 trials per checkpoint, except the velocity-control checkpoint which covers four tasks only). The checkpoints represent successive design iterations: `pi05_base` (zero-shot), `vel_100k` (100k steps, velocity control), `pos_200k` (200k steps, position control), `pos_v2` (500k steps, native 10 Hz recording, 11-dim state), and `pos_v3` (200k warm-start fine-tuning steps from `pos_v2`, augmented near-side demonstrations).

1) *Trial-Level Success Rate:* Table V reports per-task trial-level success rates across all checkpoints. A trial is successful if the specified block reaches the correct side of the workspace divider ($x > 0.70$ m for far; $x < 0.66$ m for near).

Several findings emerge from Table V. First, the zero-shot `pi05_base` baseline achieves a non-trivial 15% overall success rate (9/60 trials), with particularly strong performance on T0 (7/10, 70% success). This result — which contrasts with the naïve expectation that the pretrained model would entirely fail on an unseen embodiment — confirms that $\pi 0.5$ retains transferable manipulation priors, while also demonstrating that fine-tuning is necessary for reliable multi-task performance.

Second, a consistent far-side advantage is observed across all fine-tuned position-control checkpoints (far average: 20–60%; near average: 7–30%). Near-side carry requires the arm to move inward toward the robot base, which conflicts with the DLS null-space configuration and produces less consistent training demonstrations.

Third, green-block tasks (T4, T5) represent a persistent failure mode across all checkpoints, including `pos_v3` which was explicitly augmented with 250 near-side green demonstrations per task. The green block’s position ($y = +0.05$ m toward

TABLE V: Per-task trial-level success rate (successes / 10 trials). †: green-block tasks were not in the velocity-control training set. Best-performing checkpoint shown in bold.

Task	Instruction	pi05_base	vel 100k	pos 200k	pos_v2	pos_v3
T0	Move red block → far	7/10	0/10	1/10	6/10	9/10
T1	Move red block → near	2/10	0/10	2/10	4/10	4/10
T2	Move blue block → far	0/10	0/10	8/10	7/10	9/10
T3	Move blue block → near	0/10	0/10	0/10	5/10	4/10
T4	Move green block → far	0/10	—†	1/10	2/10	0/10
T5	Move green block → near	0/10	—†	0/10	0/10	0/10
Overall (trial-level)		9/60 (15%)	0/40 (0%)	12/60 (20%)	24/60 (40%)	26/60 (43%)
Far-side average		23%	0%	33%	50%	60%
Near-side average		7%	0%	13%	30%	27%

the robot midline) requires an arm configuration that approaches joint limits, and the **pos_v3** checkpoint shows *regression* on T4 (2/10 → 0/10) relative to **pos_v2**, suggesting that the lower learning rate warm-start did not provide sufficient gradient signal for this task. Further experiments will investigate the influence of workspace position on manipulation performance to determine whether object location contributes to the observed differences in task success.

2) Manipulation Quality: Block-Lifted Rate: Task success alone conflates true pick-and-place with accidental push success. Table VI disaggregates these by reporting the block-lifted rate, defined as the fraction of trials in which the block exceeded $z > 0.295$ m (above table height of 0.260 m plus block half-height of 0.025 m), confirming a genuine grasp-and-lift occurred.

Comparing Tables V and VI reveals an important discrepancy for the **pos_200k** checkpoint at T2 (blue → far): 8/10 task-success but only 30% lift rate, indicating that the majority of T2 successes were achieved through inadvertent pushing rather than controlled pick-and-place. Post-hoc grasp diagnostics confirmed a mean grasp error of (−155, −78, −2) mm for T2: the gripper closed 15.5 cm laterally displaced from the block, and arm translation displaced the block to the far side.

By contrast, the **pos_v2** checkpoint achieves a lift rate well above its task-success rate for T2 (90% lifted vs. 70% success), confirming genuine pick-

and-place. The **pos_v3** checkpoint shows closer agreement between lift and success rates for blue tasks (80% lifted vs. 90% success on T2; 70% vs. 40% on T3), consistent with more reliable grasp execution overall.

3) Direction Accuracy: Table VII reports the fraction of trials in which the block was displaced in the correct direction (positive x -displacement for far tasks, negative for near tasks), regardless of whether the success threshold was crossed. This metric measures task understanding independently of execution precision.

Direction accuracy reveals task understanding that is not captured by binary success metrics. The **pos_v2** checkpoint achieves 100% direction accuracy on both T0 and T2 and 80% on T4, yet its task success on T4 remains only 20%. This confirms that the policy understands which direction to move for these tasks but lacks the precision to cross the success threshold. By contrast, the **pos_v3** checkpoint regresses to 0% direction accuracy on T4 (green → far), consistent with the collapsed prediction behaviour (arm hovering at pregrasp for 600 steps) observed during diagnostic rollouts.

4) Training Metrics and Loss-Performance Relationship: Table VIII summarises the training configurations and final loss values for all fine-tuned checkpoints.

A key finding is that lower training loss does not correspond monotonically to improved task performance. The **pos_v2** checkpoint achieves a final

TABLE VI: Block-lifted rate (% of trials in which true pick-and-place was executed, i.e. $\text{block_z} > 0.295$ m during the episode). The gap between this table and Table V indicates push-based rather than pick-based success. **Bold:** highest per task.

Task	Instruction	pi05_base	vel 100k	pos 200k	pos_v2 500k	pos_v3
T0	Move red block $\rightarrow$ far	80%	0%	20%	70%	90%
T1	Move red block $\rightarrow$ near	20%	0%	30%	50%	70%
T2	Move blue block $\rightarrow$ far	0%	0%	30%	90%	80%
T3	Move blue block $\rightarrow$ near	0%	0%	30%	60%	70%
T4	Move green block $\rightarrow$ far	0%	—	10%	30%	0%
T5	Move green block $\rightarrow$ near	0%	—	10%	0%	0%
Mean lifted rate (all tasks)		17%	0%	22%	50%	45%

TABLE VII: Direction accuracy (% of trials with correct-direction block displacement) and mean signed block displacement (m; positive = toward far side). Direction accuracy measures task understanding; displacement quantifies proximity to success for near-miss trials.

Task	Instruction	Direction Accuracy (%)					Mean Displacement (m)				
		Base	Vel	Pos	V2	V3	Base	Vel	Pos	V2	V3
T0	Red $\rightarrow$ far	100	0	50	100	100	+0.127	+0.000	+0.022	+0.116	+0.159
T1	Red $\rightarrow$ near	40	0	50	40	40	−0.015	+0.000	−0.020	−0.031	−0.022
T2	Blue $\rightarrow$ far	20	0	80	100	90	+0.003	+0.000	+0.128	+0.126	+0.121
T3	Blue $\rightarrow$ near	0	0	0	50	40	+0.001	+0.000	+0.054	−0.056	−0.024
T4	Green $\rightarrow$ far	10	—	50	80	0	−0.000	—	+0.022	+0.063	+0.000
T5	Green $\rightarrow$ near	0	—	0	0	0	+0.000	—	+0.033	+0.005	+0.000

TABLE VIII: Training configuration and final metrics for each fine-tuned checkpoint.

Config	Steps	Demos	State	Loss
vel	100k	400	8D	—
pos	200k	600	8D	0.008–0.015
pos_v2	500k	600	11D	0.004
pos_v3	+200k	950	11D	<0.004

Note: All checkpoints used a peak learning rate of 2×10^{-4} except **pos_v3**, which used 5×10^{-5} .

loss of 0.004 — substantially lower than **pos_200k** (0.008–0.015) — yet achieves only marginal improvement in overall task success (40% vs. 20%). More strikingly, **pos_v2** achieves fewer successes than **pos_200k** on T2 (7/10 vs. 8/10) despite training $2.5\times$ longer. These observations indicate that the primary bottleneck is not optimisation depth but rather the quality of training data (demonstration frequency, state representation, and grasp precision). The introduction of native 10 Hz demonstration recording and end-effector Cartesian coordinates in **pos_v2** had a greater

qualitative effect on lifting behaviour (mean lifted rate 50% vs. 22%) than on raw task success.

5) *Failure Mode Analysis:* Analysis of policy roll-outs identified grasp precision as the dominant remaining failure mode. Although the policy consistently selected the correct object and generated appropriate approach trajectories, small spatial errors during gripper alignment frequently prevented successful object pickup. Additional failures included premature gripper release during transport and occasional policy collapse, in which the robot remained near the pre-grasp pose throughout the episode. Refer Table IX.

An important observation was the difference in performance between workspace regions. Tasks requiring movement towards the far side achieved higher success rates than equivalent near-side tasks, suggesting that the latter were underrepresented within the training data or more difficult to execute due to workspace geometry.

Overall, the failure analysis indicates that the

TABLE IX: Mean gripper close step (10-Hz policy steps) per task and checkpoint. Expected range from scripted demonstrations: red/green tasks: 75–107; blue tasks: 150–160. Late closure (>150) indicates the policy is approaching before closing; early closure (<20) indicates premature grasping.

Task	Base	Vel	Pos	V2	V3	Demo
T0	69	762	73	76	36	75–107
T1	77	940	110	114	64	75–107
T2	4	179	195	254	157	75–107
T3	5	183	166	147	154	75–107
T4	4	—	92	74	51	75–107
T5	4	—	81	90	60	75–107

policy has learned the high-level structure of the manipulation task but remains limited by execution accuracy rather than task understanding. These observations motivated the expanded demonstration dataset and additional fine-tuning experiments presented in the following section.

6) Zero-Shot Baseline Analysis: The `pi05_base` checkpoint result (15% overall success, 7/10 on T0) warrants explicit discussion. The original $\pi 0.5$ model has no Baxter-specific fine-tuning, yet achieves 80% block-lift rate and 100% direction accuracy on T0. This is consistent with $\pi 0.5$ ’s pre-training on large-scale diverse robot manipulation data: the model’s pretrained manipulation priors partially generalise to the Baxter workspace and camera configuration.

The performance collapse on blue and green blocks (0/10 success, near-zero lift rates) demonstrates the limits of zero-shot generalisation: the pretrained model cannot reliably distinguish between the three block colours from the current camera viewpoint, defaulting to the dominant pretraining behaviour (approaching objects in the centre of the scene, which corresponds to the red block’s position). This motivates fine-tuning as a mechanism not only for embodiment adaptation but for task routing across multiple objects with distinct spatial locations.

The VLM planner will be integrated with the manipulation policy, generating valid primitive sequences for all tested goal configurations. End-to-end collaborative evaluation remains ongoing.

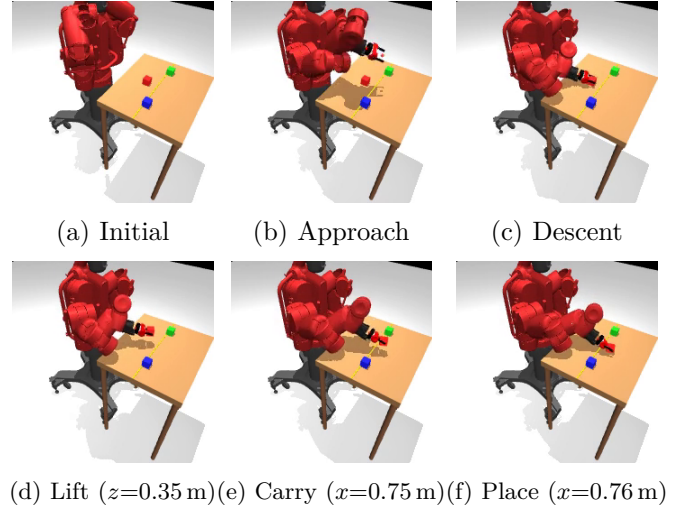


Fig. 5: Execution sequence for a successful red-block far-side trial (`pos_v3`, T0, trial 00, 9/10 success rate). The policy executes a complete pick-and-place sequence: approach, descent to within 3 mm of block centre, lift to 0.37 m, carry across the workspace divider, and controlled placement. Final block position: $x = 0.779$ m (success threshold: $x > 0.70$ m).

VI. CRITICAL REFLECTION

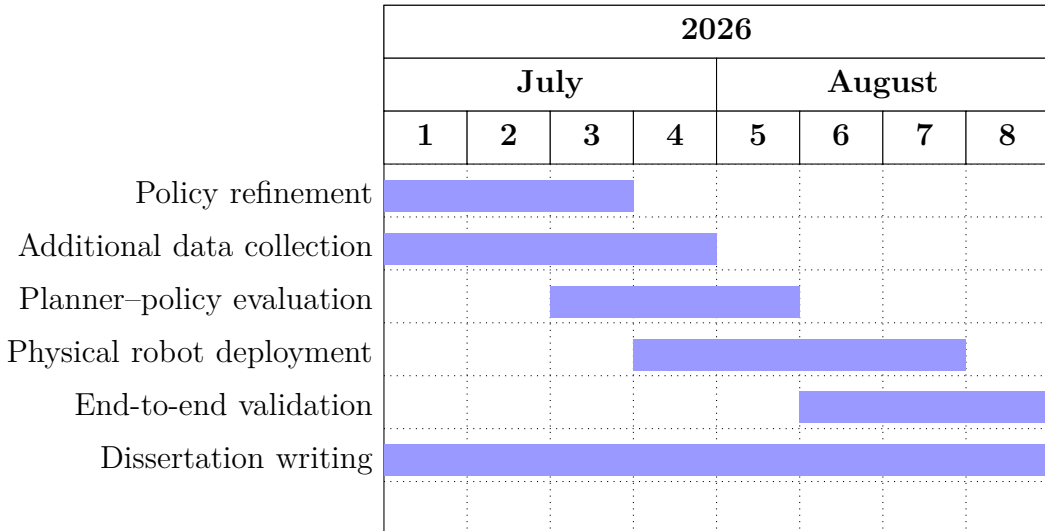
A. Evaluation of Research Progress

The current status of the project is summarised in Table X. The majority of the core technical objectives proposed during the Early Stage Gateway have been completed, including simulation environment development, demonstration collection, VLA adaptation and VLM planner development. The remaining work is primarily focused on planner–policy integration, physical robot deployment, and validation of the complete human-robot collaboration workflow.

Overall progress is aligned with the original project timeline. While several unexpected technical challenges were encountered during development, these challenges resulted in valuable insights regarding action-space design, train–inference consistency, and the practical adaptation of foundation-scale robotic policies to novel embodiments.

TABLE X: Summary of major development stages

Development Stage	Status	Outcome
Simulation Environment	Completed	Developed a custom Baxter robot environment in MuJoCo with dual-camera observations, coloured manipulation objects, and configurable task setups.
Demonstration Collection	Completed	Implemented a Damped Least Squares inverse kinematics pipeline for autonomous demonstration generation and LeRobot dataset creation.
VLA Adaptation	Completed	Fine-tuned the $\pi 0.5$ Vision-Language-Action model using LoRA and evaluated multiple policy checkpoints.
Action Representation	Completed	Redesigned the policy from velocity-control to position-control actions based on experimental evaluation.
Vision-Language Planner	Completed	Developed a Gemma-3-12B planner to generate manipulation task sequences from current and goal workspace images.
Planner-Policy Integration	In Progress	Integrated the Vision-Language planner with the fine-tuned $\pi 0.5$ policy for sequential task execution.
End-to-End System Evaluation	In Progress	Evaluating complete planner-to-policy execution, collaborative task success, and robustness prior to physical robot deployment.
Physical Robot Deployment	In Progress	Preparing the framework for fine-tuning and evaluation on the Robot DE NIRO platform.

**Fig. 6:** Planned activities for the remaining project period (8 weeks)

B. Methodological Reflection

One of the most successful methodological decisions was the adoption of a simulation-first development strategy. Initially, simulation was viewed primarily as a convenient environment for collecting demonstrations and training policies. However, the project revealed that simulation also served as an effective debugging platform, allowing numerous train-inference mismatches to be identified and resolved before deployment on

physical hardware.

The iterative development process also proved valuable. Rather than treating policy training as a one-time procedure, each generation of checkpoints was analysed in detail and used to guide subsequent modifications. The VLM planner’s evaluation was conducted separately, at an earlier stage of the project. This approach enabled systematic investigation of failure modes.

A limitation of the methodology is that evalua-

tion has primarily been conducted within simulation. Although this was appropriate for rapid experimentation, the true effectiveness of the approach can only be fully assessed following deployment on the physical Robot DE NIRO platform. Consequently, the reported results demonstrate the feasibility of the proposed approach within simulation, while validation on the physical Robot DE NIRO platform remains future work.

C. Lessons Learned

The most important lesson from this project is that successful adaptation of Vision-Language-Action models depends more on system design than optimisation alone. Checkpoint comparisons showed that improvements in manipulation performance were driven primarily by changes to the action representation and improvements in train-inference consistency rather than by longer training or lower loss values.

Finally, decomposing the task into six reusable manipulation primitives simplified both policy learning and planner integration. This modular design enables more complex collaborative behaviours to be achieved through task sequencing rather than retraining the manipulation policy.

VII. FUTURE WORK, PLANS AND RISKS

A. Remaining Work

Refer to the Gantt chart shown in Fig. 6, for an overview of the remaining work and the associated timelines. With the core system architecture complete, the remaining work focuses on evaluation and refinement rather than implementation of new components.

The first priority is improving manipulation robustness through additional policy refinement. In particular, further experiments will investigate the influence of workspace geometry on manipulation performance, as the current evaluation suggests lower success rates for objects located near the workspace boundaries, such as the green block. Future work will systematically evaluate the effect of workspace geometry on manipulation

performance by varying object positions independently of object identity, allowing the influence of workspace location to be isolated from training data imbalance.

The second priority is end-to-end evaluation of the integrated planner-policy framework, including scenarios involving human intervention and dynamic replanning.

Finally, the framework will be deployed on the Robot DE NIRO platform to evaluate transfer from simulation to physical hardware.

B. Human-Robot Collaboration Evaluation

The final dissertation will evaluate the complete planner-policy framework on collaborative rearrangement tasks. Given an initial and goal workspace configuration, the Vision-Language planner will generate a sequence of manipulation primitives which will then be executed by the fine-tuned $\pi 0.5$ policy. Performance will be measured using the collaborative task success metric defined in Section III-H3.

Additional experiments will evaluate dynamic replanning by introducing human modifications to the workspace during execution. The planner will be required to detect the updated scene, generate a revised task sequence, and continue progressing towards the desired goal configuration.

Thus, the remaining project timeline is structured around four major milestones:

- 1) Refinement and evaluation of the position-control manipulation policy.
- 2) End-to-end validation of the planner-policy architecture.
- 3) Deployment and testing on the physical Robot DE NIRO platform.
- 4) Dissertation writing, analysis, and final submission.

The final dissertation will present the complete hierarchical framework, evaluate adaptation of the $\pi 0.5$ model to a novel robot embodiment, and assess the effectiveness of integrating Vision-Language planning with Vision-Language-Action manipulation.

TABLE XI: Project risk register

Risk	Description	L	I	Risk Level	Mitigation	Contingency
Manipulation Reliability	Current policy does not achieve consistent grasping across all manipulation tasks.	4	4	16 (High)	Refine demonstration data, improve grasp trajectories, and continue checkpoint evaluation.	Collect additional demonstrations for underperforming tasks and retrain the policy.
Training Data Coverage	Limited demonstrations in some workspace regions may reduce policy generalisation, particularly near workspace boundaries.	3	4	12 (High)	Collect demonstrations across a wider range of object positions and balance the dataset.	Generate targeted demonstrations for poorly performing workspace configurations.
Sim-to-Real Transfer	Differences in sensing, dynamics, and calibration between simulation and Robot DE NIRO may reduce physical performance.	3	5	15 (High)	Validate the complete pipeline in simulation and use transferable position-control actions.	Fine-tune the policy using demonstrations collected on the physical robot.
Physical Deployment Timeline	Limited time remains for hardware validation before dissertation submission.	3	4	12 (High)	Complete software validation and end-to-end testing in simulation before deployment.	Prioritise evaluation of the core objectives and document remaining experiments as future work.
End-to-End Evaluation	Integrated planner-policy testing may reveal failure modes not observed during component-level evaluation.	2	4	8 (Medium)	Evaluate the complete planner-policy pipeline across multiple goal configurations.	Isolate planner and manipulation components to identify and resolve failures independently.

L = Likelihood (1–5), I = Impact (1–5), Risk Level = L × I.

C. Risk Assessment

Refer Table XI for the potential risks and their workarounds.

VIII. CONCLUSION

This report has presented the progress made towards developing a hierarchical human–robot collaboration framework that combines high-level Vision–Language planning with low-level Vision–Language–Action manipulation. It first motivated the need for efficient adaptation of foundation-scale VLA models to novel robotic embodiments before describing the overall system architecture, demonstration collection pipeline, and adaptation of the $\pi 0.5$ model to a Baxter robot in a custom MuJoCo environment.

The report then described the development of the fine-tuning and inference framework, including the transition from velocity-control to position-

control actions, the implementation of a Vision–Language planner for task sequencing, and the evaluation methodology used to assess policy performance. Experimental results and checkpoint comparisons were presented to analyse the impact of key engineering decisions, identify train–inference mismatches, and investigate the principal failure modes affecting manipulation performance.

Finally, the report reflected on the progress achieved since the Early Stage Gateway, discussed the remaining technical challenges, and outlined the plan for completing planner–policy integration, physical robot deployment, and end-to-end evaluation. Together, these developments establish the foundation for the final stage of the project, which will focus on validating the complete hierarchical framework for collaborative manipulation on a novel robotic platform.

IX. ACKNOWLEDGEMENT OF AI USAGE

This report was developed with the assistance of AI tools (e.g., ChatGPT, Claude) for paraphrasing and latex code editing. All ideas, implementations, and critical decisions presented in this work are my own. The use of AI tools was limited to enhancing productivity and readability, and all outputs were carefully reviewed and validated.

REFERENCES

- [1] A. Ajoudani, A. M. Zanchettin, S. Ivaldi, A. Albu-Schäffer, K. Kosuge, and O. Khatib, “Progress and prospects of the human–robot collaboration,” *Autonomous robots*, vol. 42, no. 5, 2018.
- [2] E. Matheson, R. Minto, E. G. G. Zampieri, M. Faccio, and G. Rosati, “Human–robot collaboration in manufacturing applications: A review,” *Robotics*, vol. 8, no. 4, 2019. [Online]. Available: <https://www.mdpi.com/2218-6581/8/4/100>
- [3] C. Kuang and N. Agarwal, “SAGE: A synchronized action and gaze estimation framework for comprehensive human behavior analysis,” 2025. [Online]. Available: <https://openreview.net/forum?id=AvYGy1faez>
- [4] D. Reinhardt and L. Masuda, “Therblig to robot: Action packages, robot motion and human-robot collaboration in domestic environments,” *SPOOL*, vol. 11, no. 1, pp. 49–64, 2024.
- [5] X. Chen, W. Chen, D. Lee, Y. Ge, N. Rojas, and P. Kormushev, “A backbone for long-horizon robot task understanding,” *IEEE Robotics and Automation Letters*, vol. 10, no. 2, p. 2048–2055, Feb. 2025. [Online]. Available: <http://dx.doi.org/10.1109/LRA.2025.3526441>
- [6] R. Wang, Y. Zhang, J. Mao, C.-Y. Cheng, and J. Wu, “Translating a visual lego manual to a machine-executable plan,” in *European Conference on Computer Vision*. Springer, 2022, pp. 677–694.
- [7] A. Walsman, M. Zhang, K. Kotar, K. Desingh, A. Farhadi, and D. Fox, “Break and make: Interactive structural understanding using lego bricks,” in *European Conference on Computer Vision*. Springer, 2022, pp. 90–107.
- [8] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do as i can, not as i say: Grounding language in robotic affordances,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [9] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “Rt-1: Robotics transformer for real-world control at scale,” 2023. [Online]. Available: <https://arxiv.org/abs/2212.06817>
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.15818>
- [11] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Openvla: An open-source vision-language-action model,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.09246>
- [12] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “Smolvla: A vision-language-action model for affordable and efficient robotics,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.01844>
- [13] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A vision-language-action flow model for general robot control,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.24164>
- [14] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [15] J. Wang, C. Wang, R. S. Zarrin, and Z. Erickson, “Bidirectional human-robot communication for physical human-robot interaction,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.10796>
- [16] X. Wang, X. Dengxiong, S. Bai, P. Zheng, and Y. Zhang, “Vlabot: A human vision–language–action interaction framework for robotic assembly,” *Robotics and Computer-Integrated Manufacturing*, vol. 100, p. 103268, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0736584526000475>